

1º versão

```
# Dados do Problema
pontos_entrega = [(10, 20), (30, 5), (5, 35), (20, 25), (40, 10)]
CONSUMO_POR_KM = 0.35
CAPACIDADE_TANQUE = 80

# Executando os cálculos
dist_orig = rota_original(pontos_entrega)
nova_ordem, dist_otim = rota_vizinho_mais_proximo(pontos_entrega)

# (c) Comparação de Consumo
consumo_orig = dist_orig * CONSUMO_POR_KM
consumo_otim = dist_otim * CONSUMO_POR_KM

# (d) Verificação de Reabastecimento
tanque_cheio = 80
reabastecer = consumo_otim > tanque_cheio

print(f"--- RELATÓRIO DE LOGÍSTICA ---")
print(f"Distância Original: {dist_orig:.2f} km | Consumo: {consumo_orig:.2f} L")
print(f"Distância Otimizada: {dist_otim:.2f} km | Consumo: {consumo_otim:.2f} L")
print(f"Economia Gerada: {consumo_orig - consumo_otim:.2f} L")
print(f"Nova Ordem de Entrega: {nova_ordem}")
print("-" * 30)

if reabastecer:
    print(f"ALERTA: O caminhão PRECISARÁ reabastecer. Falta: {consumo_otim - tanque_cheio:.2f} L")
else:
    print(f"STATUS: O caminhão completa a rota com um tanque. Sobra: {tanque_cheio - consumo_otim:.2f} L")
```

2º versão com gráfico

```
# Dados do Problema
pontos_entrega = [(10, 20), (30, 5), (5, 35), (20, 25), (40, 10)]
CONSUMO_POR_KM = 0.35
CAPACIDADE_TANQUE = 80

# Executando os cálculos
dist_orig = rota_original(pontos_entrega)
nova_ordem, dist_otim = rota_vizinho_mais_proximo(pontos_entrega)

# (c) Comparação de Consumo
consumo_orig = dist_orig * CONSUMO_POR_KM
consumo_otim = dist_otim * CONSUMO_POR_KM
```

```

# (d) Verificação de Reabastecimento
tanque_cheio = 80
reabastecer = consumo_otim > tanque_cheio

print(f"--- RELATÓRIO DE LOGÍSTICA ---")
print(f"Distância Original: {dist_orig:.2f} km | Consumo: {consumo_orig:.2f} L")
print(f"Distância Otimizada: {dist_otim:.2f} km | Consumo: {consumo_otim:.2f} L")
print(f"Economia Gerada: {consumo_orig - consumo_otim:.2f} L")
print(f"Nova Ordem de Entrega: {nova_ordem}")
print("-" * 30)

if reabastecer:
    print(f"ALERTA: O caminhão PRECISARÁ reabastecer. Falta: {consumo_otim - tanque_cheio:.2f} L")
else:
    print(f"STATUS: O caminhão completa a rota com um tanque. Sobra: {tanque_cheio - consumo_otim:.2f} L")

import matplotlib.pyplot as plt

# Prepare data for plotting
x_orig = [p[0] for p in pontos_entrega]
y_orig = [p[1] for p in pontos_entrega]

x_otim = [p[0] for p in nova_ordem]
y_otim = [p[1] for p in nova_ordem]

plt.figure(figsize=(10, 8))

# Plot original route
plt.plot(x_orig, y_orig, 'o-', label=f'Rota Original (Dist: {dist_orig:.2f} km)', color='blue', alpha=0.7)

# Plot optimized route
plt.plot(x_otim, y_otim, 'x--', label=f'Rota Otimizada (Dist: {dist_otim:.2f} km)', color='red', alpha=0.7)

# Label points
for i, ponto in enumerate(pontos_entrega):
    plt.text(ponto[0] + 1, ponto[1] + 1, f'P{i+1}', fontsize=10)

plt.title('Comparação de Rotas de Entrega')
plt.xlabel('Coordenada X')
plt.ylabel('Coordenada Y')
plt.grid(True)
plt.legend()
plt.gca().set_aspect('equal', adjustable='box')
plt.show()

```

3º versão com gasto de combustível

```
# Dados do Problema
pontos_entrega = [(10, 20), (30, 5), (5, 35), (20, 25), (40, 10)]
CONSUMO_POR_KM = 0.35
CAPACIDADE_TANQUE = 80

# Executando os cálculos
dist_orig = rota_original(pontos_entrega)
nova_ordem, dist_otim = rota_vizinho_mais_proximo(pontos_entrega)

# (c) Comparação de Consumo
consumo_orig = dist_orig * CONSUMO_POR_KM
consumo_otim = dist_otim * CONSUMO_POR_KM

# (d) Verificação de Reabastecimento
tanque_cheio = 80
reabastecer = consumo_otim > tanque_cheio

print(f"--- RELATÓRIO DE LOGÍSTICA ---")
print(f"Distância Original: {dist_orig:.2f} km | Consumo: {consumo_orig:.2f} L")
print(f"Distância Otimizada: {dist_otim:.2f} km | Consumo: {consumo_otim:.2f} L")
print(f"Economia Gerada: {consumo_orig - consumo_otim:.2f} L")
print(f"Nova Ordem de Entrega: {nova_ordem}")
print("-" * 30)

if reabastecer:
    print(f"ALERTA: O caminhão PRECISARÁ reabastecer. Falta: {consumo_otim - tanque_cheio:.2f} L")
else:
    print(f"STATUS: O caminhão completa a rota com um tanque. Sobra: {tanque_cheio - consumo_otim:.2f} L")

import matplotlib.pyplot as plt

# Prepare data for plotting
x_orig = [p[0] for p in pontos_entrega]
y_orig = [p[1] for p in pontos_entrega]

x_otim = [p[0] for p in nova_ordem]
y_otim = [p[1] for p in nova_ordem]

plt.figure(figsize=(10, 8))

# Plot original route
plt.plot(x_orig, y_orig, 'o-', label=f'Rota Original (Dist: {dist_orig:.2f} km)', color='blue', alpha=0.7)

# Plot optimized route
```

```

plt.plot(x_otim, y_otim, 'x--', label=f'Rota Otimizada (Dist:
{dist_otim:.2f} km)', color='red', alpha=0.7)

# Label points
for i, ponto in enumerate(pontos_entrega):
    plt.text(ponto[0] + 1, ponto[1] + 1, f'P{i+1}', fontsize=10)

plt.title('Comparação de Rotas de Entrega')
plt.xlabel('Coordenada X')
plt.ylabel('Coordenada Y')
plt.grid(True)
plt.legend()
plt.gca().set_aspect('equal', adjustable='box')
plt.show()

# New plot for fuel consumption
plt.figure(figsize=(8, 6))
consumo_labels = ['Rota Original', 'Rota Otimizada']
consumo_valores = [consumo_orig, consumo_otim]

plt.bar(consumo_labels, consumo_valores, color=['blue', 'red'],
alpha=0.7)
plt.ylabel('Consumo de Combustível (L)')
plt.title('Comparação de Consumo de Combustível por Rota')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

```

4º versão com valor do combustivel

```

# Dados do Problema
pontos_entrega = [(10, 20), (30, 5), (5, 35), (20, 25), (40, 10)]
CONSUMO_POR_KM = 0.35
CAPACIDADE_TANQUE = 80
PRECO_DIESEL_POR_LITRO = 7.28 # Preço do diesel por litro em R$

# Executando os cálculos
dist_orig = rota_original(pontos_entrega)
nova_ordem, dist_otim = rota_vizinho_mais_proximo(pontos_entrega)

# (c) Comparação de Consumo
consumo_orig = dist_orig * CONSUMO_POR_KM
consumo_otim = dist_otim * CONSUMO_POR_KM

# (d) Verificação de Reabastecimento
tanque_cheio = 80
reabastecer = consumo_otim > tanque_cheio

# (e) Cálculo do Custo em Reais
custo_orig = consumo_orig * PRECO_DIESEL_POR_LITRO

```

```

custo_otim = consumo_otim * PRECO_DIESEL_POR_LITRO

print(f"--- RELATÓRIO DE LOGÍSTICA ---")
print(f"Distância Original: {dist_orig:.2f} km | Consumo: {consumo_orig:.2f} L | Custo: R$ {custo_orig:.2f}")
print(f"Distância Otimizada: {dist_otim:.2f} km | Consumo: {consumo_otim:.2f} L | Custo: R$ {custo_otim:.2f}")
print(f"Economia Gerada (Litros): {consumo_orig - consumo_otim:.2f} L")
print(f"Economia Gerada (Reais): R$ {custo_orig - custo_otim:.2f}")
print("-" * 30)

if reabastecer:
    print(f"ALERTA: O caminhão PRECISARÁ reabastecer. Falta: {consumo_otim - tanque_cheio:.2f} L")
else:
    print(f"STATUS: O caminhão completa a rota com um tanque. Sobra: {tanque_cheio - consumo_otim:.2f} L")

import matplotlib.pyplot as plt

# Prepare data for plotting
x_orig = [p[0] for p in pontos_entrega]
y_orig = [p[1] for p in pontos_entrega]

x_otim = [p[0] for p in nova_ordem]
y_otim = [p[1] for p in nova_ordem]

plt.figure(figsize=(10, 8))

# Plot original route
plt.plot(x_orig, y_orig, 'o-', label=f'Rota Original (Dist: {dist_orig:.2f} km)', color='blue', alpha=0.7)

# Plot optimized route
plt.plot(x_otim, y_otim, 'x--', label=f'Rota Otimizada (Dist: {dist_otim:.2f} km)', color='red', alpha=0.7)

# Label points
for i, ponto in enumerate(pontos_entrega):
    plt.text(ponto[0] + 1, ponto[1] + 1, f'P{i+1}', fontsize=10)

plt.title('Comparação de Rotas de Entrega')
plt.xlabel('Coordenada X')
plt.ylabel('Coordenada Y')
plt.grid(True)
plt.legend()
plt.gca().set_aspect('equal', adjustable='box')
plt.show()

# Plot for fuel consumption comparison
plt.figure(figsize=(8, 6))

```

```
consumo_labels = ['Rota Original', 'Rota Otimizada']
consumo_valores = [consumo_orig, consumo_otim]

plt.bar(consumo_labels, consumo_valores, color=['blue', 'red'],
alpha=0.7)
plt.ylabel('Consumo de Combustível (L)')
plt.title('Comparação de Consumo de Combustível por Rota')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

# New plot for fuel cost comparison
plt.figure(figsize=(8, 6))
custo_labels = ['Rota Original', 'Rota Otimizada']
custo_valores = [custo_orig, custo_otim]

plt.bar(custo_labels, custo_valores, color=['blue', 'red'], alpha=0.7)
plt.ylabel('Custo de Combustível (R$)')
plt.title('Comparação de Custo de Combustível por Rota')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```